

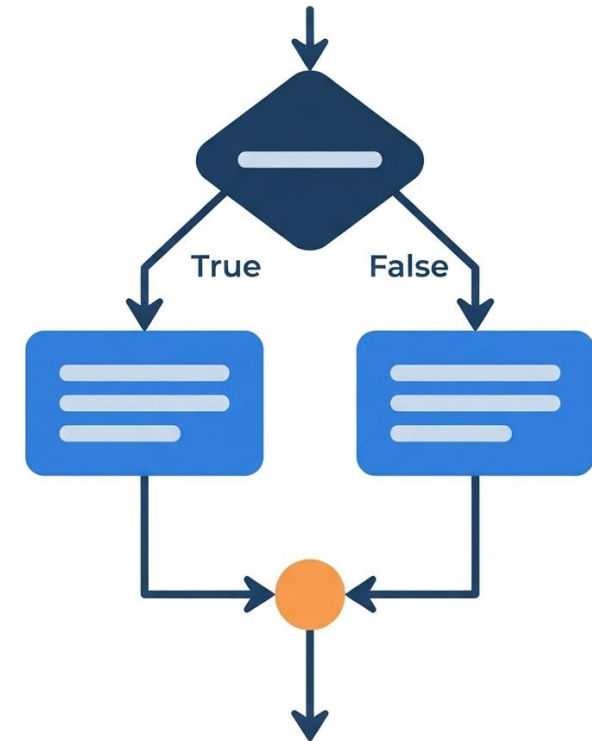
Проекти з розгалуженнями. Повна форма

Тема 3. Алгоритми та програми · § 4.6, с. 134–139

Коротка форма ВІДСІЮЄ. Повна форма ВИБИРАЄ

Якщо так — одне. Якщо ні — інше.

- if <логічний вираз>:
 - <послідовність команд 1>
- else:
 - <послідовність команд 2>
- ★ Виконається РІВНО ОДНА з двох гілок. Не жодна, не обидві. Завжди одна.
- ⚠ else — із двокрапкою, з тілом із відступом, і сам стоїть РІВНО на рівні if.



Три помилки з else

- else без двокрапки → `SyntaxError: expected ':'`
- else без if вище → `SyntaxError: invalid syntax`
- тіло else без відступу → `IndentationError: expected an indented block after 'else' statement`
- ★ else — НЕ всередині if. Він ПОРУЧ із ним, рівно на тому самому рівні.
- Не глибше. Не лівіше.
- ⚠ Це критично: на наступному уроці рівнів стане три.

Два розв'язки однієї задачі

- У домашній роботі був пункт 3: чим розв'язок підручника відрізняється від нашого?
- ★ У нас — `return`. У книжці — `else`. Ось вони поруч.
- Однакові рядки: `if a - 3*b == 0: i showinfo(...)`
- Різниця: у нас `return i z = ...` БЕЗ відступу. У книжці `else: i z = ...` З відступом.
- Обидва запускаються з 6, 2 → однакове повідомлення. З 2, 1 → однакове -4.0
- ★ Два різні коди. Однакова поведінка. ОБИДВА ПРАВИЛЬНІ.

[Місце для ілюстрації]

ТІЛЬКИ СКРІНШОТ: два вікна редактора IDLE поруч, одним кадром. Ліворуч `vyraz_if.py` із виділеним рядком `return`, праворуч `vyraz_else.py` із виділеним рядком `else`:

Чому в повній формі `return` не потрібен?

- У короткій формі після розгалуження код ІДЕ ДАЛІ. Тому й потрібен `return` — «зупинись».
- ★ У повній формі йти нікуди: або одне, або інше. Гілки взаємовиключні.
- Якщо виконалася гілка `if` — гілка `else` не виконається ВЗАГАЛІ.
- Спробуємо дописати `return` у `else...` і нічого не змінилося.
- Він тут не шкодить. Він просто НІЧОГО НЕ РОБИТЬ.
- ★ Коротка форма ВІДСІЮЄ. Повна форма ВИБИРАЄ. Це різні інструменти.

Три перевірки чи два варіанти?

- ТРИ захисні перевірки підряд (порожнє поле, ще одне, нуль) → ★ коротка форма з return
- Бо повна дала б else у else у else — три рівні відступів. Нечитабельно.
- РІВНО ДВА рівноправні варіанти («вистачило» / «не вистачило») → ★ повна форма
- Бо в обох випадках треба ЩОСЬ СКАЗАТИ. Обидві гілки потрібні, return нікуди не веде.
- ★ Правило: відсіюєш — коротка. Вибираєш — повна.

Марійка, версія 3: відповідає завжди

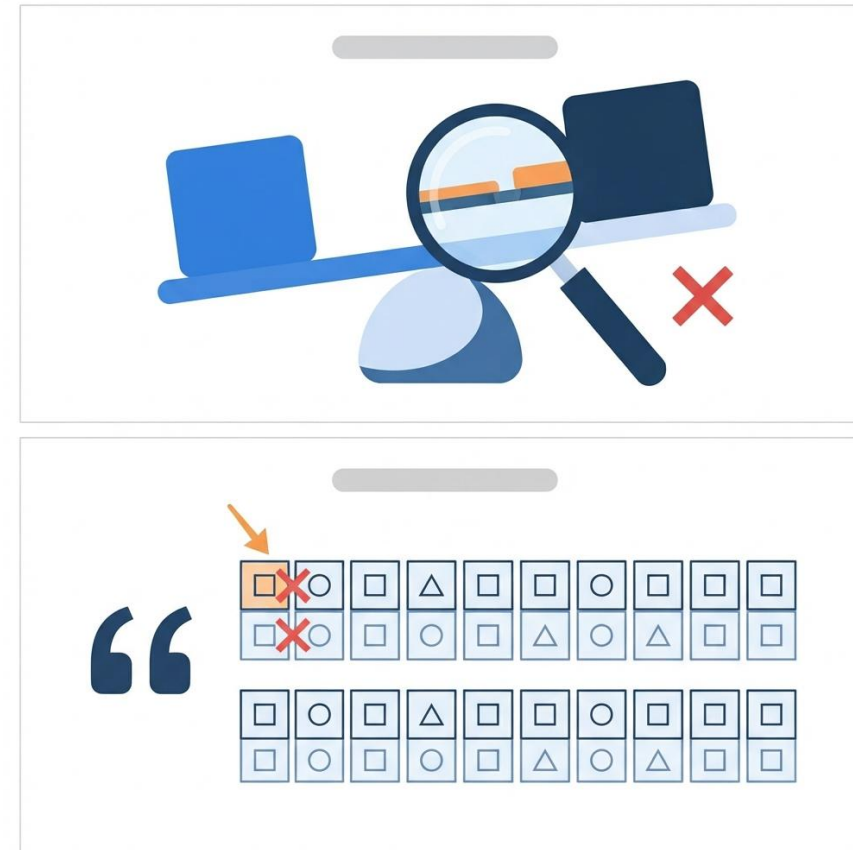
- `if r >= 0:`
- `label5['text'] = 'Вистачило! Залишиться: ' + str(round(r, 2)) + ' грн'`
- `else:`
- `label5['text'] = 'Не вистачило на ' + str(round(-r, 2)) + ' грн'`
- ★ У другій гілці — `-r`, а не `r`. Мінус уже сказаний СЛОВОМ «не вистачило».
- ★ Умова `r >= 0`, а не `r > 0`. Грошей рівно стільки, скільки треба — це ВИСТАЧИЛО.

Набір, якого раніше не було

- Набір 1: 200 / 150 / 32 / 25 → $r = 51$ зелений, «Вистачило»
- Набір 2: 100 / 200 / 30 / 28 → $r = -68$ червоний, «Не вистачило на 68»
- ★ Набір 3: 149 / 150 / 32 / 25 → $r = 0$ РІВНО. Що покаже проєкт?
- Замінімо $\geq$ на $>$... і набір 3 дає «Не вистачило на 0.0 грн». Прочитайте вГОЛОС.
- ★ ОДИН СИМВОЛ. І проєкт бреше — але тільки в одному випадку з мільйона.
- ⚠ Набори 1 і 2 цього не показали б НІКОЛИ. Тільки набір НА МЕЖІ.

Дві пастки порівняння

- $0.1 + 0.2 == 0.3 \rightarrow \text{False (!)}$. $0.3 - 3 * 0.1 \rightarrow -5.551115123125783\text{e-}17$
- ⚠ Задача 1 із $a = 0.3$, $b = 0.1$: перевірка на нуль НЕ спрацює.
- Проєкт поділить на мізерне число й видасть -9007199254740992.0 . Помилки НЕМАЄ.
- $'10' > '9' \rightarrow \text{False (!)}$ Тексти порівнюються по символах, як слова в словнику.
- ⚠ Забули `float()` — і 10 виявиться меншим за 5. Розгалуження працює, відповідь ні.
- ★ Порівняння з ЧИСЛОМ дає чесний `TypeError`. Порівняння ТЕКСТІВ — молчить.



Скільки тестових наборів і які

- розгалуження на 2 гілки $\rightarrow$ щонайменше 2, по одному на гілку
- порівняння $\geq$ або $\leq$ $\rightarrow$ ★ ПЛЮС набір рівно на межі
- ділення на вираз $\rightarrow$ плюс набір, де дільник $= 0$
- квадрат у знаменнику, як $(x^2 - 9)$ $\rightarrow$ ⚠ ДВА нулі: 3 і -3
- Другий нуль забувають завжди. $x^2 = 9$ при 3 і при -3 .
- ★ Правило з уроку 13, тепер із доказом: тестуй МЕЖУ, а не середину.

Домашнє завдання

- 1. § 4.6, с. 134–135 і 137–139. ДВІ блок-схеми поруч у зошиті + одне речення різниці. Фото.
- 2. Марійка, версія 3 → `marijka_else.py`. ⚠ ТРИ скріншоти, третій — набір 149.
- На третьому напис мусить бути ЗЕЛЕНИМ. Червоний = у вас > замість > =
- 3. Задача 1 повною формою → `zavdannia_4_6_1_else.py`, return прибрати.
- Плюс одне речення: ЧОМУ return більше не потрібен. Це головне питання уроку.
- 4. (за бажанням) Перевірити пастки в Shell. Строк — до 09.12.